

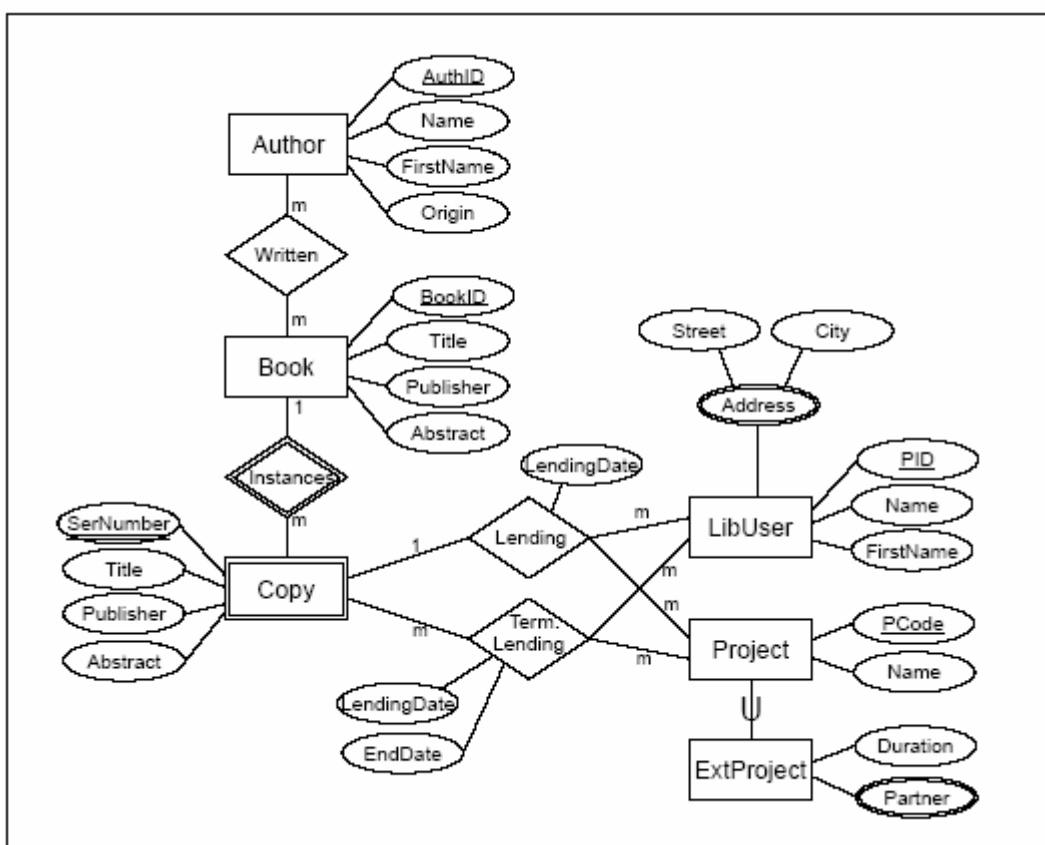
บทที่ 7

การแปลงอีอาร์ไดอะแกรมไปเป็นสคีมาของฐานข้อมูลแบบเชิงวัตถุ

ในการสร้างสคีมาของฐานข้อมูลเชิงวัตถุ เราสามารถทำได้ 2 แบบหลักๆ คือ 1. การแปลงจากตัวสคีมาของฐานข้อมูลแบบเชิงสัมพันธ์ไปเป็นสคีมาของฐานข้อมูลแบบเชิงวัตถุ 2. การแปลงจากอีอาร์ไดอะแกรมไปเป็นสคีมาของฐานข้อมูลแบบเชิงวัตถุ โดยวิธีแรกเป็นการใช้วิธีทางคณิตศาสตร์เข้ามาช่วยสร้างสคีมาของคลาสในฐานข้อมูลแบบเชิงวัตถุ ส่วนวิธีที่สองเป็นการสร้างคลาสโดยพิจารณาจากเอ็นติตี้โทพของอีอาร์ไดอะแกรม ซึ่งวิธีนี้เราจะได้ผลลัพธ์ของคลาสที่สมบูรณ์กว่าวิธีแรก

7.1. การแปลงจากตัวสคีมาของฐานข้อมูลแบบเชิงสัมพันธ์

ในวิธีการนี้ เราต้องมีความรู้ในเรื่องของอีอาร์ไดอะแกรมอยู่บ้าง โดยต้องรู้วิธีการแปลงจากโมเดลอีอาร์ไปเป็นสคีมาของฐานข้อมูลแบบเชิงวัตถุ



รูปที่ 7.1 ตัวอย่าง ER Diagram

7.1.1. อีอีอาร์โมเดล (EER model)

อีอีอาร์โมเดลเป็นการแสดงเอนติตี้ไทป์ และความสัมพันธ์ระหว่างเอนติตี้ไทป์ ซึ่งได้ขยายเพิ่มความสัมพันธ์บางอย่างมาจากอีอาร์โมเดล อย่างเช่น ความสัมพันธ์แบบการสืบทอดคุณสมบัติจากเอนติตี้ไทป์พ่อ(Supertype) มายังเอนติตี้ไทป์ลูก (Subtype) โดยความสัมพันธ์แบบนี้เราใช้คำว่า “is a” แทนการสืบทอดคุณสมบัติ ตัวอย่าง ถ้าเอนติตี้ไทป์ E2 เป็นเอนติตี้ไทป์ลูกของเอนติตี้ไทป์ E1 แล้วแอททริบิวต์และรีเลชันต่างๆ ที่มีอยู่ในเอนติตี้ไทป์ E1 จะถูกถ่ายทอดมาให้กับเอนติตี้ไทป์ E2

7.1.2. รีเลชันนอลโมเดล (Relational model)

เอนติตี้และรีเลชันชิพระหว่างเอนติตี้เป็นรูปแบบโมเดลของรีเลชัน โดยแต่ละรีเลชันมาจากการกำหนดกลุ่มของแอททริบิวต์ และส่วนรีเลชันชิพระหว่างเอนติตี้จะมีการใช้งานกำหนดแอททริบิวต์ร่วมกัน ความสัมพันธ์ที่สำคัญอย่างหนึ่งคือ กฎข้อบังคับความสัมพันธ์ภายในรีเลชันหรือเรียกว่า กฎข้อบังคับในการอ้างอิง(Referential integrity) หมายความว่า ถ้าแอททริบิวต์ A ของรีเลชัน R1 ถูกกำหนดให้เป็นคีย์นอก (foreign key) โดยอ้างอิงถึงคีย์ในรีเลชัน R2 แล้วทุกค่าของแอททริบิวต์ A ต้องมีค่าตรงกับค่าคีย์ที่อยู่ในรีเลชัน R2

7.1.3. ออบเจกต์-ออเรียนเตดโมเดล (Object-oriented model)

ในรูปแบบโมเดลเชิงวัตถุ มีการกำหนดคลาสในรูปแบบของแอททริบิวต์ ซึ่งเป็นตัวอธิบายโครงสร้างของตัวออบเจกต์ในคลาส และมีเมธอดเป็นตัวอธิบายพฤติกรรมของออบเจกต์

โดยแอททริบิวต์ในคลาสของโมเดลเชิงวัตถุจะสนับสนุนชนิดของข้อมูลได้หลากหลายกว่าโมเดลเชิงสัมพันธ์ โดยจะมีชนิดข้อมูล เช่น integer, char, byte, character of string เป็นต้น รวมถึงชนิดของข้อมูลที่ซับซ้อน ซึ่งมาจากการที่เราประกาศคลาส หรือชนิดของข้อมูลที่มาจากการรวมชนิดของข้อมูลต่างๆ มาประกอบกันเรียกว่า composite attribute

คลาสในรูปแบบโมเดลเชิงวัตถุ นั้น สามารถจัดการได้แบบคลาสลำดับชั้น (Class hierarchy) โดยคลาสลูก (Subclass) จะสืบทอดคุณสมบัติต่างๆ มาจากคลาสพ่อของมัน (Superclass)

7.1.4. การแปลงรีเลชันนอลสคีมาไปเป็นออบเจกต์-ออเรียนเตดสคีมา

เป็นวิธีการแปลงรีเลชันนอลสคีมาไปเป็นออบเจกต์-ออเรียนเตดสคีมาโดยการใช้คณิตศาสตร์ระดับสูงในการคำนวณ ซึ่งมีกฎหลักๆ อยู่ 7 ข้อ (มาจากเอกสารของ Weiyi Meng, Aqueo Kamada, Yu-His Chang) แต่กล่าวโดยคร่าวๆ ได้ว่า

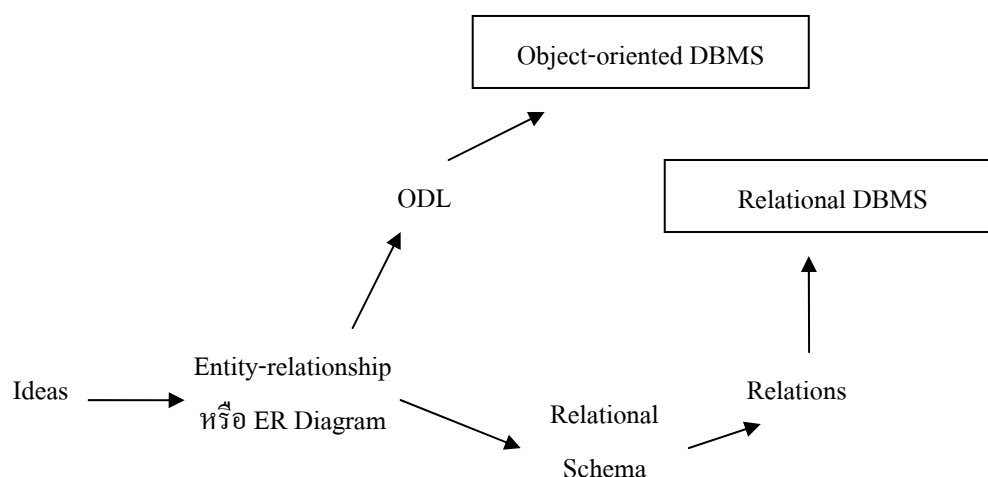
1. กำหนดประเภทของรีเลชันนอลสคีมาเพื่อใช้เป็นข้อมูลในการแปลงเป็นคลาส ซึ่งต้องได้รับข้อมูลเพิ่มเติมจากผู้ระบบเพื่อแบ่งแยกประเภทของรีเลชันนอลสคีมา
2. แปลงรีเลชันนอลสคีมาไปเป็นคลาส
3. นำรีเลชันนอลสคีมาหรือแอททริบิวต์ในแต่ละรีเลชันนอลสคีมาแปลงไปเป็นแอททริบิวต์ของแต่ละคลาส

โดยในการแปลงรีเลชันนอลสคีมามาเป็นออปเจก-ออเรียนเตดสคีมานั้น มีปัญหาหลักๆ ด้วยกัน 2 ข้อที่เราต้องนำมาพิจารณาในการแปลงสคีมามา

1. การแปลงอ็อบเจกต์ไดอะแกรมไปยังรีเลชันนอลสคีมามาทำให้รูปแบบประโยค(Semantic) บางอย่างหายไป ทำให้เมื่อเราต้องการทำการแปลงรีเลชันนอลสคีมามาเป็นออปเจก-ออเรียนเตดสคีมามาหรืออ็อบเจกต์ไดอะแกรม จำเป็นต้องได้รับข้อมูลเพิ่มเติมบางอย่าง
2. รีเลชันนอลสคีมามาที่ทำการแปลงควรจะได้รับการออกแบบมาอย่างดี หรือ ควรจะผ่านคุณสมบัติของนอร์มัลฟอร์มที่ 3 (3NF) เป็นอย่างน้อย

7.2. การแปลงจากอ็อบเจกต์ไดอะแกรม

เป็นวิธีการที่ใช้ในการนำโครงสร้างของอ็อบเจกต์ไดอะแกรมมาใช้ในการพิจารณาสร้างเป็นคลาสในออปเจก-ออเรียนเตดสคีมามา ซึ่งวิธีการนี้เราจะได้คุณสมบัติต่างๆ ของคลาสและความสัมพันธ์ต่างๆ ระหว่างคลาสได้อย่างถูกต้องตามหลักการของออปเจก-ออเรียนเตดสคีมามา



รูปที่ 7.2 โมเดลของฐานข้อมูลและกระบวนการใช้งาน

จากรูปที่ 7-2 เป็นการแสดงกระบวนการออกแบบ โดยเราเริ่มจากหาแนวความคิดเกี่ยวกับข้อมูลที่ต้องการสร้างเป็นโมเดล แนวความคิดเหล่านั้นจะถูกแปลให้อยู่ในรูปของอ็อบเจกต์ไดอะแกรม แล้วหลังจากนั้นก็นำอ็อบเจกต์ไดอะแกรมที่ได้แปลงเป็นรูปแบบภาษาที่ใช้ออกแบบโครงสร้างของฐานข้อมูลได้ 2 อย่างหลักๆ คือ รีเลชันนอลสคีมามา หรือภาษากำหนดเชิงวัตถุ (Object Definition Language : ODL) โดยหลังจากแปลงเป็นรีเลชันนอลสคีมามาก็จะทำการแปลให้อยู่ในรูปของรีเลชันนอล โดยใช้งานกับระบบฐานข้อมูลแบบเชิงสัมพันธ์ (Relational DBMS : RDBMS) หรือหลังจากแปลงเป็นภาษากำหนดเชิงวัตถุเพื่อนำไปใช้งานกับระบบฐานข้อมูลแบบเชิงวัตถุ(Object-oriented DBMS : OODBMS)

การแปลงจากอ็อบเจกต์ไดอะแกรมไปเป็นภาษากำหนดเชิงวัตถุ นั้นมีส่วนหลักๆ ในการพิจารณา 3 ส่วน

1. กลุ่มของเอนิตีในอ็อบเจกต์ไดอะแกรม ซึ่งถูกแปลงให้เป็นคลาสในภาษากำหนดเชิงวัตถุ

2. แอททริบิวต์ ซึ่งใช้เป็นค่าในการอธิบายคุณสมบัติของคลาส โดยแอททริบิวต์ในอ็อบเจกต์ไคอะแกรม และในภาษากำหนดเชิงวัตถุ จะนำมาจากแนวคิดเดียว ก็คือ กำหนดแอททริบิวต์ของเอนิตีในไคอะแกรมให้เป็นแอททริบิวต์ของคลาสในภาษากำหนดเชิงวัตถุ

3. ความสัมพันธ์ระหว่างเอนิตี ซึ่งใช้ในการแสดงความสัมพันธ์ระหว่างเอนิตีตั้งแต่ 2 เอนิตีขึ้นไป โดยความสัมพันธ์ระหว่างเอนิตีในอ็อบเจกต์ไคอะแกรม กับความสัมพันธ์ของคลาสในภาษากำหนดเชิงวัตถุ จะมีลักษณะคล้ายๆ กัน แต่อย่างไรก็ตาม

(a) ในอ็อบเจกต์ไคอะแกรม เรากำหนดให้ชื่อแอททริบิวต์ให้ใช้แสดงความสัมพันธ์ระหว่างเอนิตี แต่ในขณะที่ความสัมพันธ์ของคลาสในภาษากำหนดเชิงวัตถุจะกำหนดโดยใช้คำว่า “relationship” และประกาศความสัมพันธ์ย้อนกลับด้วยคำว่า “inverse” ให้กับคลาสในภาษากำหนดเชิงวัตถุ ดังแสดงในรูปที่ 7-3

Interface Movie

```
{
    Attribute String title;
    Attribute Integer year;
    Attribute Integer length;
    Attribute enum Film { color, blackAndWhite } filmType;
    Relationship Set<Star> stars
        Inverse Star::starredIn;
    Relationship Studio ownedBy
        Inverse Studio::owns;
}
```

Interface Star

```
{
    Attribute String name;
    Attribute Struct Addr
        { String street, String city } address;
    Relationship Set<Movie> starredIn
        Inverse Movie::stars;
}
```

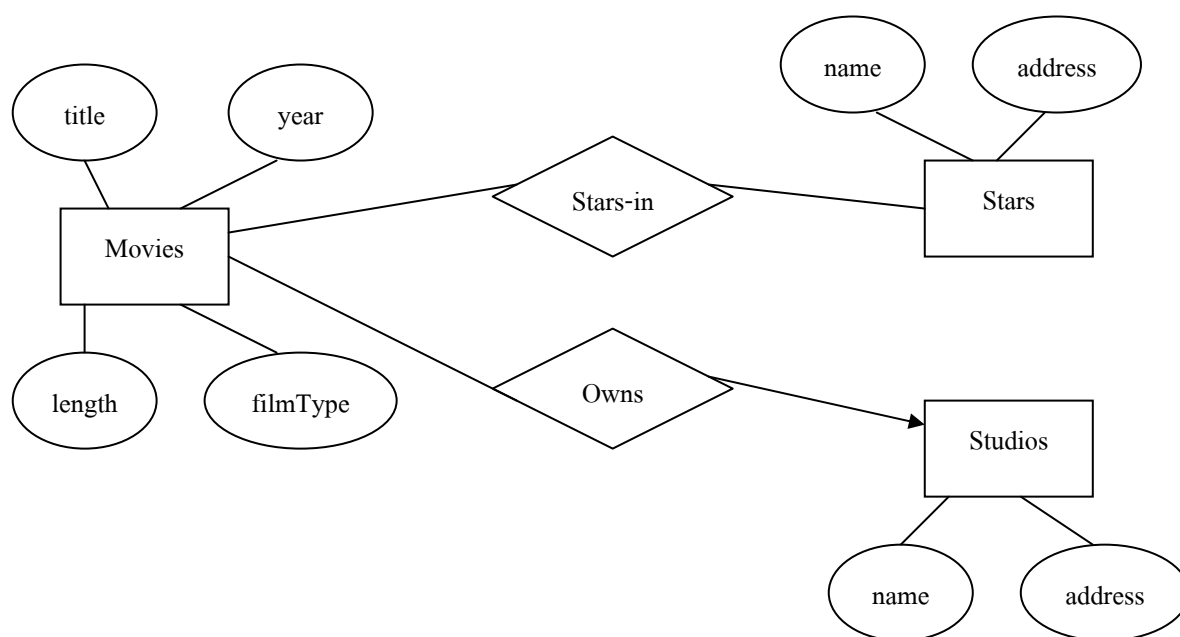
รูปที่ 7.3 แสดงตัวอย่างรูปแบบของความสัมพันธ์ต่างๆ

(b) ในอ็อบเจกต์ไคอะแกรม ความสัมพันธ์ระหว่างเอนิตีแบบหลายเอนิตี (Many-to-many relationship) นั้นเวลาแปลงเป็นรีเลชันนอลสกีมา เราจะแปลงได้ 3 รีเลชัน ก็จะมีรีเลชันของเอนิตี 2 รีเลชัน และมีรีเลชันที่สร้างจากคีย์หลักของทั้งสองเอนิตีอีก 1 รีเลชัน แต่ในภาษากำหนดเชิงวัตถุ เราจะแปลงได้เป็น 2 คลาสเท่านั้น คือ 2 คลาสของเอนิตีในอ็อบเจกต์ไคอะแกรม และทั้งสองคลาสต้องมีการกำหนดความสัมพันธ์

ระหว่างกันด้วย โดยใช้คำว่า “relationship” ในการประกาศความสัมพันธ์ ตัวอย่างในรูปที่ 7-4 เราได้จากความสัมพันธ์ระหว่างคลาส Movie กับคลาส Star ซึ่งเป็นความสัมพันธ์แบบหลายคลาส โดยในคลาส Movie ใช้ตัวแปร stars ในการแสดงความสัมพันธ์ระหว่าง และในคลาส Star มีตัวแปร starredIn เพื่อประกาศเป็นความสัมพันธ์ย้อนกลับของตัวแปร stars ในคลาส Movie

(c) ความสัมพันธ์ในอ็อบเจกต์ไคอะแกรมสามารถแสดงความสัมพันธ์ระหว่างเอนทิตีได้มากกว่า 2 เอนทิตี แต่ในความสัมพันธ์ของคลาสในภาษากำหนดเชิงวัตถุจะแสดงความสัมพันธ์ระหว่างคลาสได้เพียง 2 คลาสเท่านั้น ซึ่งหมายความว่า ภาษากำหนดเชิงวัตถุนี้ไม่สนับสนุนความสัมพันธ์แบบยูนารี (U-nary relationship)

ตัวอย่างการแปลงความสัมพันธ์ของคลาสในภาษากำหนดเชิงวัตถุ



รูปที่ 7.4 เป็นการแสดงความสัมพันธ์ระหว่างเอนทิตีในอ็อบเจกต์ไคอะแกรม

เราสามารถแปลงเป็นคลาสในภาษากำหนดเชิงวัตถุได้ ดังนี้คือ

Interface Movie

{

Attribute String title;

Attribute Integer year;

Attribute Integer length;

Attribute enum Film { color, blackAndWhite } filmType;

Relationship Set<Star> stars

Inverse Star::starredIn;

```

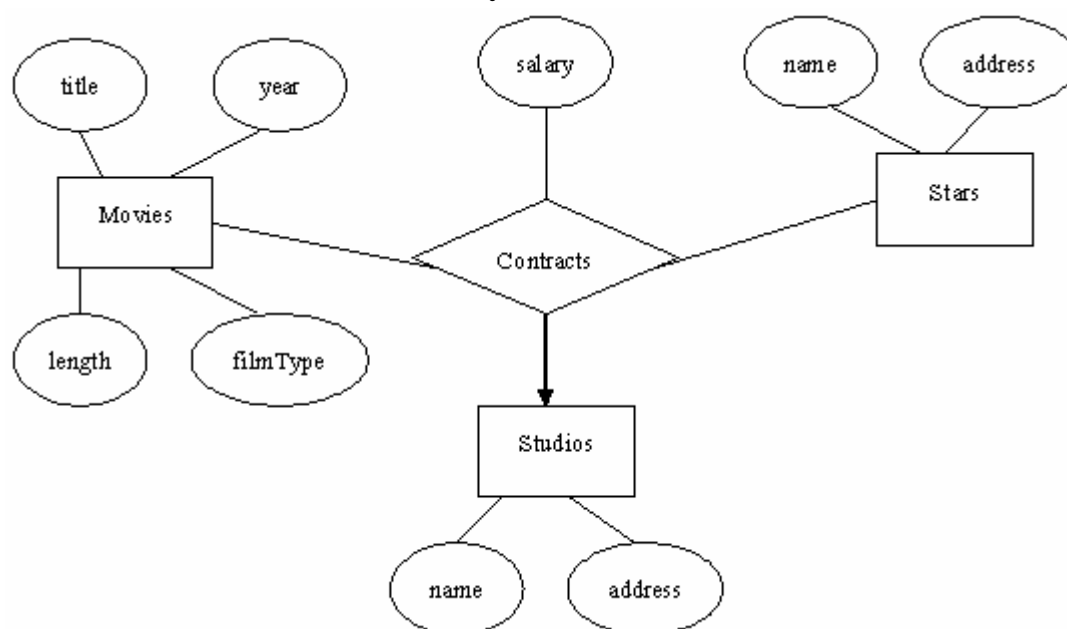
Relationship Studio ownedBy
    Inverse Studio::owns;
}

Interface Star
{
    Attribute String name;
    Attribute Struct Addr
        { String street, String city } address;
    Relationship Set<Movie> starredIn
        Inverse Movie::stars;
}

Interface Studio
{
    Attribute String name;
    Attribute String address;
    Relationship Set<Movie> owns
        Inverse Movie::owndBy;
}

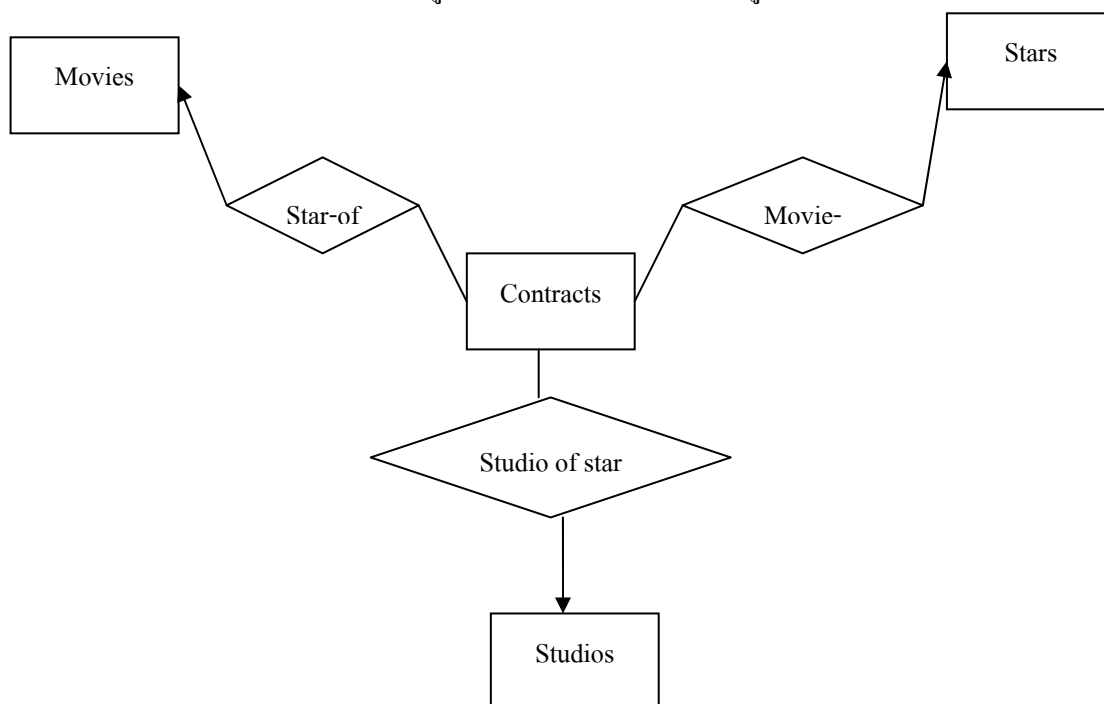
```

ตัวอย่างการแปลงความสัมพันธ์แบบยูนารี(U-nary relationship)



รูปที่ 7.5 แสดงความสัมพันธ์ระหว่างเอนทิตีในอีอาร์ไดอะแกรมแบบยูนารี

เราต้องแปลงรีเลชัน Contract ให้กลายเป็นคลาสก่อน จากนั้นก็สร้างความสัมพันธ์ระหว่างคลาส Contract กับคลาสทั้ง 3 คลาส ได้แก่ คลาส Movie, คลาส Studio และคลาส Star และต้องมีการประกาศรีเลชันชิฟให้ไปชี้ที่คลาสทั้ง 3 ด้วย ดังตัวอย่างในรูปที่ 7-6 และตัวอย่างคลาสในรูปที่ 7-7



รูปที่ 7.6 แสดงรูปอ็อบเจกต์โปรแกรมใหม่ที่เราได้แปลงรีเลชัน *Contract* เป็นเอนทิตี

Interface Movie

```

{
    Attribute String title;
    Attribute Integer year;
    Attribute Integer length;
    Attribute enum Film { color, blackAndWhite } filmType;
    Relationship Set<Contract> contracts
        Inverse Contract::movieOfStar;
}
  
```

Interface Star

```

{
    Attribute String name;
    Attribute Struct Addr
        { String street, String city } address;
    Relationship Set<Contract> starredIn
        Inverse Contract::stars;
}
  
```

Interface Studio

```
{
    Attribute String name;
    Attribute String address;
    Relationship Set<Contract> owns
        Inverse Contract::ownerOfStar;
}
```

Interface Contract

```
{
    Attribute Integer salary;
    Relationship Movie movieOfStar
        Inverse Movie::contracts;
    Relationship Star stars
        Inverse Star::starredIn;
    Relationship Studio ownerOfStar
        Inverse Studio::owns;
}
```

รูปที่ 7.7 เป็นคลาสในภาษากำหนดเชิงวัตถุที่ได้จากการแปลงรูปที่ 7.6